NetroTrack Production — Configuration & YAML Appendix

Copy/paste reference for Docker Compose, Dockerfile OpenSSL fix, GCP LB and redirect configuration

1. Purpose

This appendix is the copy/paste reference for the configuration artifacts used in the working setup. It intentionally excludes real secret values. Replace variables through the VM's protected .env file.

2. Production docker-compose.yml

```yaml
services:
  postgres:
    image: postgres:16
    container_name: netrotrack-postgres
    restart: unless-stopped
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    volumes:
      - ./postgres/data:/var/lib/postgresql/data
    ports:
      - "127.0.0.1:5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 10s
      timeout: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    container_name: netrotrack-redis
    restart: unless-stopped
    command:
      - redis-server
      - --requirepass
      - ${REDIS_PASSWORD}
      - --maxmemory
      - 768mb
      - --maxmemory-policy
      - allkeys-lru
    ports:
      - "127.0.0.1:6379:6379"
    healthcheck:
      test: ["CMD", "redis-cli", "-a", "${REDIS_PASSWORD}", "ping"]
      interval: 10s
      timeout: 5s
      retries: 5

  backend:
    image: ghcr.io/netrofusionlabs/netro_track-backend:production
    container_name: netrotrack-backend
    restart: unless-stopped
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
    environment:
      NODE_ENV: production
      PORT: 3000
      DATABASE_URL: postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@postgres:5432/${POSTGRES_DB}?schema=public
      REDIS_URL: redis://:${REDIS_PASSWORD}@redis:6379
      JWT_ACCESS_SECRET: ${JWT_ACCESS_SECRET}
      JWT_REFRESH_SECRET: ${JWT_REFRESH_SECRET}
      LOG_LEVEL: ${LOG_LEVEL:-info}
      R2_ACCOUNT_ID: ${R2_ACCOUNT_ID}
      R2_ACCESS_KEY_ID: ${R2_ACCESS_KEY_ID}
      R2_SECRET_ACCESS_KEY: ${R2_SECRET_ACCESS_KEY}
      R2_BUCKET_NAME: ${R2_BUCKET_NAME}
      R2_PUBLIC_URL: ${R2_PUBLIC_URL}
      GOOGLE_MAPS_API_KEY: ${GOOGLE_MAPS_API_KEY}
    expose:
      - "3000"
```

3. Prisma/OpenSSL Dockerfile pattern

```
FROM node:20-bookworm-slim AS builder

RUN apt-get update     && apt-get install -y --no-install-recommends openssl     && rm -rf /var/lib/apt/lists/*

# dependency installation
# Prisma generation
# application build


FROM node:20-bookworm-slim AS runner

RUN apt-get update     && apt-get install -y --no-install-recommends openssl     && rm -rf /var/lib/apt/lists/*

# copy production artifacts
# start application
```

The key requirement discovered during production deployment is that OpenSSL must exist during Prisma generation and at runtime.

4. HTTP redirect URL map

```
name: netrotrack-http-redirect-map

defaultUrlRedirect:
  httpsRedirect: true
  stripQuery: false
```

Create/import:

```
cat > /tmp/netrotrack-http-redirect.yaml <<'EOF'
name: netrotrack-http-redirect-map

defaultUrlRedirect:
  httpsRedirect: true
  stripQuery: false
EOF

gcloud compute url-maps import netrotrack-http-redirect-map   --project=netro-track-prod   --source=/tmp/netrotrack-http-redirect.ya
```

5. Complete LB command set

```
# Static IP
gcloud compute addresses describe netrotrack-prod-ip   --project=netro-track-prod --global   --format="get(address)"

# VM network/tag
gcloud compute instances describe netro-track-prod   --project=netro-track-prod --zone=us-central1-a   --format="yaml(name,networkIn

gcloud compute instances add-tags netro-track-prod   --project=netro-track-prod --zone=us-central1-a   --tags=netrotrack-backend

# Firewall
gcloud compute firewall-rules create netrotrack-allow-lb   --project=netro-track-prod --network=default   --direction=INGRESS --acti

# Instance group
gcloud compute instance-groups unmanaged create netrotrack-prod-ig   --project=netro-track-prod --zone=us-central1-a

gcloud compute instance-groups unmanaged add-instances netrotrack-prod-ig   --project=netro-track-prod --zone=us-central1-a   --inst

gcloud compute instance-groups unmanaged set-named-ports netrotrack-prod-ig   --project=netro-track-prod --zone=us-central1-a   --na

# Health check
gcloud compute health-checks create http netrotrack-backend-health   --project=netro-track-prod --port=3000   --request-path=/health
```

6. Backend service verification

```
gcloud compute backend-services describe netrotrack-backend-service   --project=netro-track-prod --global   --format="yaml(name,prot

gcloud compute backend-services get-health netrotrack-backend-service   --project=netro-track-prod --global   --format="yaml"
```

7. HTTPS certificate / proxy / forwarding rule

```
gcloud compute ssl-certificates create netrotrack-api-cert   --project=netro-track-prod --global   --domains=netro-track-api.netrofu

gcloud compute ssl-certificates describe netrotrack-api-cert   --project=netro-track-prod --global   --format="yaml(name,type,manage

gcloud compute target-https-proxies create netrotrack-prod-https-proxy   --project=netro-track-prod   --url-map=netrotrack-prod-url-
```

```
gcloud compute forwarding-rules create netrotrack-prod-https-rule   --project=netro-track-prod --global   --target-https-proxy=netro
```

8. HTTP redirect proxy / forwarding rule

```
gcloud compute target-http-proxies create netrotrack-prod-http-redirect-proxy   --project=netro-track-prod   --url-map=netrotrack-ht
```

```
gcloud compute forwarding-rules create netrotrack-prod-http-rule   --project=netro-track-prod --global   --target-http-proxy=netrotr
```

9. Final verification commands

```
dig +short netro-track-api.netrofusion.in
```

```
nc -vz 8.232.197.48 443
```

```
curl -I https://netro-track-api.netrofusion.in/health
```

```
curl -IL http://netro-track-api.netrofusion.in/health
```

```
openssl s_client   -connect netro-track-api.netrofusion.in:443   -servername netro-track-api.netrofusion.in
```

```
gcloud compute forwarding-rules list   --project=netro-track-prod --global   --format="table(name,IPAddress,IPProtocol,portRange,tar
```

10. Operational notes

• Keep the reserved global IP. Do not recreate it casually because DNS points to it.
• Keep the VM's backend network tag and firewall rule aligned.
• Do not expose PostgreSQL or Redis publicly.
• Keep application secrets only in the VM environment/secret-management mechanism; never commit them to Git.
• For a production redeploy, verify the new image, container status, /health, backend-service health, and public HTTPS endpoint in that order.
• The current backend service uses an unmanaged instance group containing the single production VM; this is not autoscaling.